

Конкурентност - две изчисления нямат ясно дефинирана последователност на изпълнение / възможно е да споделят ресурс.

Паралелизъм - две изчисления се изпълняват едновременно  
 ↳ предполага се, че има много процесори, за да може задачите да вървят паралелно.

Конкурентност в Python?

отделни процеси

нишки

зелени нишки

нашия код се грижи за предаването на контрол

системни нишки

операционната система се грижи за предаването на контрол

GIL - механизъм, който позволява само една нишка да изпълнява байткод в даден момент  
 ако има много нишки, те се изпълняват последователно

CPU-bound - тоест "обвързан с процесора"

↳ изпълнението на задача / програма е ограничено главно от скоростта на централния процесор

IO-bound - "обвързан с входно-изходните операции"

fork → създава копие на програмата

↳ така операционната система реализира паралелизъм

+

-

стабилност  
 синът и бащата са независими

количеството между процеси е  
 създаването на нов  
 труден

## Създаване на нова нишка

- нишка - наследник на `threading.Thread`
- код - в метода `run`
- създаване и изпълнение на кода паралелно - с метода `start`

## Критични секции

- части от кода, които могат да бъдат изпълнени само от една нишка/процес в даден момент

## Семафор

`threading.Lock()` → връща `Lock` обект

`acquire()` → само ние притежаваме този `Lock`

`release()` → освобождава `Lock`-а и може да се заключи от друг с `acquire()`

Ако `Lock`-а е зает и викнем `acquire()` → метода чака, докато не се освободи

Всички `Lock`-подобни обекти са и `context manager`-и

При използване с `with`: 1) викаме `acquire()` преди блока

2) викаме `release()` след блока

`threading.Semaphore(k)` → връща семафор с интерфейс като на `Lock` и стойност `k`

`acquire()` → стойността се намалява с 1

`release()` → стойността се увеличава с 1

Ако `k=0`, `acquire()` спи, докато някой не увеличи с `release()`

`Lock() ≡ Semaphore(1)`

`threading.Event()` → няколко нишки изискват събитие от друга

`wait()` → блокира докато събитието не се случи

`set()` → "случва" събитието

`threading.local()` → позволява свободен достъп до атрибути